

GOODREADS BOOKS & REVIEWS ANALYSIS

USING HADOOP MAPREDUCE AND APACHE PIG

Big Data Lab Report

Prepared by

Abrar Faisal

ID

2104010202286

Course

CSE 4346 – Big Data Analytics Lab

September 15, 2026

Table of Contents

1. Problem Statement	3
2. Dataset Description	3
3. Summary of Data Analysis	4
4. MapReduce Analysis	5
4.1 Task 1: Rating Histogram	6
4.2 Task 2: Average Rating by Publication Year	8
4.3 Task 3: Book–Review Join	14
5. Pig Analysis	19
5.1 Task 1: Books by Language	19
5.2 Task 2: Popular & Highly Rated Books.....	20
5.3 Task 3: Rating Histogram	22
5.4 Task 4: Book–Review Join	23
5.5 Task 5: Average Rating by Publication Decade.....	25
6. Data Quality Notes	29
7. Conclusion.....	29

1. Problem Statement

The goal of this project is to analyze a Goodreads-style book catalog and a sample of user reviews using the Hadoop ecosystem — specifically MapReduce (Java) and Apache Pig — in order to extract meaningful insights across several dimensions of the data, including rating distribution, publication trends over time, the relationship between books and their reviews, language, popularity, and decade of publication. Three MapReduce jobs and five Pig scripts were designed, implemented, and executed against the two datasets on a single-node Hadoop cluster.

2. Dataset Description

This project uses two related datasets: a book metadata catalog (books.csv) and a sample of user reviews (reviews_100k.csv), joined on book_id where a task calls for it. Both are comma-separated with a header row.

Fields — books.csv

Field	Description
book_id	An integer that uniquely identifies the book
title	A string containing the book's title
authors	A string naming one or more authors
average_rating	A floating-point number giving the book's average rating on Goodreads
ratings_count	An integer giving the number of ratings the book had received
publication_year	An integer giving the publication year (missing for ~25% of rows — see §6)
language_code	A string giving the language/edition code, e.g. eng, en-US, spa (missing for ~38% of rows)
isbn	A string giving the book's ISBN (missing for some rows)

Fields — reviews_100k.csv

Field	Description
user_id	A string that uniquely identifies the reviewer
book_id	An integer identifying the book being reviewed (joins to books.csv)
review_id	A string that uniquely identifies the review
rating	An integer star rating, 0–5 (0 denotes "shelved / read but not rated" on Goodreads)
review_text	The free-text body of the review
date_added	Timestamp the review was added to the user's shelf
date_updated	Timestamp the review was last updated
read_at	Timestamp the user finished reading (blank if not yet finished)
started_at	Timestamp the user started reading (blank if not yet started)
n_votes	An integer count of "helpful" votes the review received
n_comments	An integer count of comments left on the review

Dataset Statistics

Metric	Value
books.csv records	219,235
reviews_100k.csv records	100,000 (sampled down from a 500,000-row source)
books.csv size on disk	≈ 15.0 MB (15,756,275 bytes)
reviews_100k.csv size on disk	≈ 77.9 MB (81,716,160 bytes)
Distinct books with ≥1 review	42,345 — all 42,345 are present in books.csv (100% join coverage)
Storage location	HDFS: /lab/input/ (Java jobs); local working directory (Pig jobs)

3. Summary of Data Analysis

The following MapReduce design patterns are demonstrated across the three Java jobs:

- **Summarization:** Task 1 counts how many reviews fall into each star rating, 0 through 5, and reuses its own Reducer as the Combiner
- **Summarization with a Combiner:** Task 2 accumulates a running "sum:count" string per publication_year on the map side, then divides only once, at the very end, in the Reducer
- **Reduce-side Join:** Task 3 combines books.csv and reviews_100k.csv on book_id using MultipleInputs, tagging each mapper's output ("B|title" vs. "R|rating") so the single Reducer can tell which dataset a value came from

The following three MapReduce analyses were implemented:

- **Rating Histogram:** Count of reviews at each star rating, 0–5
- **Average Rating by Publication Year:** Average books.csv rating, grouped by publication_year
- **Book–Review Join:** For every book with at least one review — title, review count, and average review rating

The following five Pig analyses were implemented:

- **Books by Language:** Book count and average rating per language_code, top 15 languages
- **Popular & Highly Rated Books:** Books with average_rating ≥ 4.5 and ratings_count ≥ 1,000, top 20 by rating count
- **Rating Histogram:** Same question as MapReduce Task 1, used as a cross-check
- **Book–Review Join:** Same question as MapReduce Task 3, via a single Pig JOIN, filtered to books with at least 20 reviews, top 25 by average review rating
- **Average Rating by Publication Decade:** Average books.csv rating and book count, grouped by decade

4. MapReduce Analysis

All three jobs share a single utility class, `CsvUtils` — a dependency-free, RFC-4180-style CSV line parser. It is needed because `review_text` and some book titles contain commas and quotation marks that a naive `value.toString().split(",")` would mis-parse, shifting every field after the split point.

Shared Utility Class (*CsvUtils.java*)

```
import java.util.ArrayList;
import java.util.List;

/**
 * Minimal RFC-4180-style CSV line parser.
 *
 * Handles:
 * - fields wrapped in double quotes
 * - commas embedded inside quoted fields (e.g. book titles like
 *   "Crown Royal (Crown John Mystery, #13)")
 * - escaped double quotes inside quoted fields (" -> ")
 *
 * Assumes exactly one CSV record per input line, which is true for both
 * books.csv and reviews_100k.csv (verified: no embedded newlines in any
 * field, so Hadoop's default line-based TextInputFormat is safe to use).
 *
 * Compile this together with the job classes, e.g.:
 * javac -classpath $(hadoop classpath) -d classes CsvUtils.java RatingHistogram.java
 */
public final class CsvUtils {

    private CsvUtils() {
    }

    public static String[] parseCsvLine(String line) {
        List<String> fields = new ArrayList<String>();
        StringBuilder current = new StringBuilder();
        boolean inQuotes = false;

        for (int i = 0; i < line.length(); i++) {
            char c = line.charAt(i);
            if (inQuotes) {
                if (c == '"') {
                    if (i + 1 < line.length() && line.charAt(i + 1) == '"') {
                        current.append('"');
                        i++; // skip the escaped quote
                    } else {
                        inQuotes = false;
                    }
                } else {
                    current.append(c);
                }
            } else {
                if (c == '"') {
                    inQuotes = true;
                } else if (c == ',') {
                    fields.add(current.toString());
                    current.setLength(0);
                } else {
                    current.append(c);
                }
            }
        }
        fields.add(current.toString());
        return fields.toArray(new String[0]);
    }
}
```

4.1 Task 1: Rating Histogram

Counts how many reviews fall into each star rating (0–5)

This analysis comprises of:

- **RatingMapper:** Emits <rating, 1> for every review row with a parseable rating
- **SumReducer:** Sums the per-rating counts; this same class is also registered as the job's Combiner

Source (*RatingHistogram.java*)

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/**
 * Java Task 1: Rating histogram.
 *
 * Input : reviews_100k.csv
 * (user_id,book_id,review_id,rating,review_text,date_added,date_updated,
 * read_at,started_at,n_votes,n_comments)
 *
 * Output: rating <TAB> number_of_reviews
 *
 * A word-count-style job: introduces InputFormat handling, header skipping,
 * a Combiner, and CSV-safe field extraction (review_text itself may contain
 * commas and quotes, so the naive value.toString().split(",") would break).
 */
public class RatingHistogram {

    public static class RatingMapper extends Mapper<Object, Text, Text, IntWritable> {
        private static final IntWritable ONE = new IntWritable(1);
        private final Text ratingKey = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("user_id,book_id,review_id,rating")) {
                return; // header row
            }
            String[] fields = CsvUtils.parseCsvLine(line);
            if (fields.length < 4) {
                return;
            }
            String rating = fields[3].trim();
            if (rating.isEmpty()) {
                return;
            }
            ratingKey.set(rating);
            context.write(ratingKey, ONE);
        }
    }

    public static class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
        private final IntWritable result = new IntWritable();

        @Override
        protected void reduce(Text key, Iterable<IntWritable> values, Context context)
```

```

        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable v : values) {
            sum += v.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: RatingHistogram <input reviews_100k.csv on HDFS> <output
dir>");
        System.exit(2);
    }
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Rating Histogram");
    job.setJarByClass(RatingHistogram.class);
    job.setMapperClass(RatingMapper.class);
    job.setCombinerClass(SumReducer.class);
    job.setReducerClass(SumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

Command: `hadoop jar rating-histogram.jar RatingHistogram /lab/input/reviews_100k.csv /lab/output/task1`

Execution log (excerpt):

```

INFO input.FileInputFormat: Total input files to process : 1
INFO mapreduce.JobSubmitter: number of splits:1
INFO impl.YarnClientImpl: Submitted application application_1789307359180_0004
INFO mapreduce.Job: Running job: job_1789307359180_0004
INFO mapreduce.Job: Job job_1789307359180_0004 running in uber mode : false
INFO mapreduce.Job:  map 0% reduce 0%
INFO mapreduce.Job:  map 100% reduce 0%
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_1789307359180_0004 completed successfully

```

Job Counters (selected):

Map input records	100,001
Map output records	100,000
Combine output records	6
Reduce input groups	6
Reduce output records	6
Bytes Written	45

Results

Rating	Number of Reviews
0	2,488
1	2,903
2	7,927
3	23,082
4	35,659
5	27,941

```
C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\books-reviews-lab>hdfs dfs -ls /lab/output/task1
Found 2 items
-rw-r--r-- 1 ABRAR supergroup      0 2026-09-13 20:05 /lab/output/task1/_SUCCESS
-rw-r--r-- 1 ABRAR supergroup    45 2026-09-13 20:05 /lab/output/task1/part-r-00000

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\books-reviews-lab>hdfs dfs -cat /lab/output/task1/part-r-00000
0      2488
1      2903
2      7927
3     23082
4     35659
5     27941
```

Figure 4.1 — `hdfs dfs -cat` output for Rating Histogram

Note: All six rating buckets sum to exactly 100,000 (2,488 + 2,903 + 7,927 + 23,082 + 35,659 + 27,941) — matching "Map output records" in the job counters exactly, confirming no rows were lost or duplicated between the map and reduce phases.

4.2 Task 2: Average Rating by Publication Year

Average books.csv rating, grouped by publication_year

This analysis comprises of:

- **YearMapper:** Parses `publication_year` and `average_rating` from each `books.csv` row, emitting `<year, "rating:1">` only when the year field is a valid 4-digit value and the rating parses as a number
- **SumCountCombiner:** Locally sums the "sum:count" pairs for a given year on each map node before the shuffle
- **AverageReducer:** Sums all incoming "sum:count" pairs and divides once, at the very end, to compute the average

Source (*AvgRatingByYear.java*)

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/**
 * Java Task 2: Average book rating by publication year.
```

```

*
* Input : books.csv
*   (book_id,title,authors,average_rating,ratings_count,publication_year,
*   language_code,isbn)
*
* Output: publication_year <TAB> average_rating <TAB> number_of_books
*
* Demonstrates why you cannot just average partial averages: both the
* Combiner and the Reducer emit "sum:count" pairs, and only the final
* Reducer divides sum by count. Rows with a blank/invalid year or rating
* are dropped (about 25% of books.csv has no publication_year).
*/
public class AvgRatingByYear {

    public static class YearMapper extends Mapper<Object, Text, Text, Text> {
        private final Text yearKey = new Text();
        private final Text sumCount = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("book_id,title,authors")) {
                return; // header row
            }
            String[] fields = CsvUtils.parseCsvLine(line);
            if (fields.length < 6) {
                return;
            }

            String year = fields[5].trim();
            if (year.isEmpty() || !year.matches("\\d{4}")) {
                return;
            }

            double rating;
            try {
                rating = Double.parseDouble(fields[3].trim());
            } catch (NumberFormatException e) {
                return;
            }

            yearKey.set(year);
            sumCount.set(rating + ":1");
            context.write(yearKey, sumCount);
        }
    }

    /** Pre-aggregates sum/count pairs on the mapper side. */
    public static class SumCountCombiner extends Reducer<Text, Text, Text, Text> {
        private final Text out = new Text();

        @Override
        protected void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
            double sum = 0.0;
            long count = 0L;
            for (Text v : values) {
                String[] parts = v.toString().split(":");
                sum += Double.parseDouble(parts[0]);
                count += Long.parseLong(parts[1]);
            }
            out.set(sum + ":" + count);
            context.write(key, out);
        }
    }

    /** Final aggregation: turns the accumulated sum/count into an average. */

```

```

public static class AverageReducer extends Reducer<Text, Text, Text, Text> {
    private final Text out = new Text();

    @Override
    protected void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        double sum = 0.0;
        long count = 0L;
        for (Text v : values) {
            String[] parts = v.toString().split(":");
            sum += Double.parseDouble(parts[0]);
            count += Long.parseLong(parts[1]);
        }
        double avg = (count == 0) ? 0.0 : sum / count;
        out.set(String.format("%.3f\t%d", avg, count));
        context.write(key, out);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: AvgRatingByYear <input books.csv on HDFS> <output dir>");
        System.exit(2);
    }
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Rating By Year");
    job.setJarByClass(AvgRatingByYear.class);
    job.setMapperClass(YearMapper.class);
    job.setCombinerClass(SumCountCombiner.class);
    job.setReducerClass(AverageReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

Execution

Command: `hadoop jar avg-rating-by-year.jar AvgRatingByYear /lab/input/books.csv /lab/output/task2`

Execution log (excerpt):

```

INFO input.FileInputFormat: Total input files to process : 1
INFO mapreduce.JobSubmitter: number of splits:1
INFO impl.YarnClientImpl: Submitted application application_1789307359180_0005
INFO mapreduce.Job: Running job: job_1789307359180_0005
INFO mapreduce.Job: Job job_1789307359180_0005 running in uber mode : false
INFO mapreduce.Job:  map 100% reduce 0%
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_1789307359180_0005 completed successfully

```

Job Counters (selected):

Map input records	219,236
Map output records	164,664
Combine output records	148
Reduce input groups	148
Reduce output records	148
Bytes Written	2,108

Results

148 distinct publication years appear in the output; the table below shows a representative sample — the earliest years, a run of recent high-volume years, and the two implausible years discussed in §6. The full 148-row result is in the project’s output/ folder and shown in full across Figures 4.2a–4.2c.

Year	Avg. Rating	Book Count
1213	4.020	1
1393	4.130	1
1394	3.880	1
1395	3.907	3
1396	3.810	1
1833	3.240	1
1839	3.850	1
1843	4.220	1
1850	3.710	1
1887	4.150	1
⋮	⋮	⋮
2000	3.777	2,239
2005	3.794	4,303
2010	3.796	9,096
2011	3.793	12,080
2104	4.046	7
2105	4.260	3
2911	4.000	1
3013	4.020	2

```
C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\books-reviews-lab>hdfs dfs -cat /lab/output/task2/part-r-00000
1213 4.020 1
1393 4.130 1
1394 3.880 1
1395 3.907 3
1396 3.810 1
1833 3.240 1
1839 3.850 1
1843 4.220 1
1850 3.710 1
1887 4.150 1
1890 3.950 3
1892 3.926 5
1893 3.700 3
1895 3.280 1
1899 3.255 2
1900 4.053 3
1901 4.220 1
1902 4.090 3
1903 3.427 3
1904 3.810 2
1906 3.535 2
1907 3.683 4
1908 3.765 4
1909 3.640 4
1910 3.583 3
1912 3.360 3
1913 3.548 4
1914 4.040 1
1915 3.670 4
1916 3.800 3
1917 3.637 6
1918 3.328 5
1919 3.660 1
1920 3.799 7
1921 3.390 3
1922 3.717 6
1923 3.681 7
1924 3.753 3
1925 3.514 10
1926 3.623 9
1927 3.451 8
1928 3.576 22
```

Figure 4.2 — `hdfs dfs -cat` output for Average Rating by Year

Note: A few entries show implausible publication years — 1213, 2911, and 3013 — that pass the "exactly 4 digits" validity check without being plausible years. See Section 6, Data Quality Notes, for the full investigation, including a plausible non-Gregorian-calendar explanation for a related cluster of years in the 1390s.

4.3 Task 3: Book–Review Join

For every book with at least one review — title, review count, and average review rating

This analysis comprises of:

- **BooksMapper:** Emits `<book_id, "B|title">` for every `books.csv` row
- **ReviewsMapper:** Emits `<book_id, "R|rating">` for every `reviews_100k.csv` row
- **JoinReducer:** A classic reduce-side join: for each `book_id`, separates the tagged values by source, then emits title, review count, and average rating — only for books that received at least one review

The two mappers read from two different files at once via `MultipleInputs`, so this single job performs the join without any separate Pig or SQL step.

Source (*BookReviewJoin.java*)

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
```

```

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.MultipleInputs;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/**
 * Java Task 3: Reduce-side join of books.csv and reviews_100k.csv on book_id.
 *
 * For every book that has at least one review, outputs:
 *   book_id <TAB> title <TAB> review_count <TAB> average_review_rating
 *
 * Two Mappers read the two different files and tag each value so the single
 * Reducer can tell which side it came from once both are shuffled together
 * on the same book_id key: "B|<title>" for books.csv, "R|<rating>" for
 * reviews_100k.csv. This is the classic reduce-side join pattern, done here
 * with MultipleInputs instead of a Pig JOIN, to show the same logic at the
 * MapReduce level.
 */
public class BookReviewJoin {

    public static class BooksMapper extends Mapper<Object, Text, Text, Text> {
        private final Text bookIdKey = new Text();
        private final Text tagged = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("book_id,title,authors")) {
                return; // header row
            }
            String[] fields = CsvUtils.parseCsvLine(line);
            if (fields.length < 2) {
                return;
            }
            String bookId = fields[0].trim();
            String title = fields[1].trim();
            if (bookId.isEmpty()) {
                return;
            }
            bookIdKey.set(bookId);
            tagged.set("B|" + title);
            context.write(bookIdKey, tagged);
        }
    }

    public static class ReviewsMapper extends Mapper<Object, Text, Text, Text> {
        private final Text bookIdKey = new Text();
        private final Text tagged = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("user_id,book_id,review_id,rating")) {
                return; // header row
            }
            String[] fields = CsvUtils.parseCsvLine(line);
            if (fields.length < 4) {
                return;
            }
            String bookId = fields[1].trim();
            String rating = fields[3].trim();
            if (bookId.isEmpty() || rating.isEmpty()) {
                return;
            }
        }
    }
}

```

```

        bookIdKey.set(bookId);
        tagged.set("R|" + rating);
        context.write(bookIdKey, tagged);
    }
}

public static class JoinReducer extends Reducer<Text, Text, Text, Text> {
    private final Text out = new Text();

    @Override
    protected void reduce(Text bookId, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        String title = null;
        double sum = 0.0;
        long count = 0L;

        for (Text v : values) {
            String s = v.toString();
            if (s.startsWith("B|")) {
                title = s.substring(2);
            } else if (s.startsWith("R|")) {
                try {
                    sum += Double.parseDouble(s.substring(2));
                    count++;
                } catch (NumberFormatException ignored) {
                    // skip malformed rating value
                }
            }
        }

        if (count == 0) {
            return; // only emit books that actually have reviews
        }
        double avg = sum / count;
        out.set((title == null ? "UNKNOWN_TITLE" : title)
            + "\t" + count + "\t" + String.format("%.3f", avg));
        context.write(bookId, out);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 3) {
        System.err.println("Usage: BookReviewJoin <books.csv on HDFS> <reviews_100k.csv on HDFS> <output dir>");
        System.exit(2);
    }
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Book Review Reduce-Side Join");
    job.setJarByClass(BookReviewJoin.class);

    MultipleInputs.addInputPath(job, new Path(args[0]), TextInputFormat.class, BooksMapper.class);
    MultipleInputs.addInputPath(job, new Path(args[1]), TextInputFormat.class,
ReviewsMapper.class);

    job.setReducerClass(JoinReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileOutputFormat.setOutputPath(job, new Path(args[2]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

Command: `hadoop jar book-review-join.jar BookReviewJoin /lab/input/books.csv /lab/input/reviews_100k.csv /lab/output/task3`

Execution log (excerpt):

```
INFO input.FileInputFormat: Total input files to process : 1
INFO input.FileInputFormat: Total input files to process : 1
INFO mapreduce.JobSubmitter: number of splits:2
INFO impl.YarnClientImpl: Submitted application application_1789307359180_0006
INFO mapreduce.Job: Running job: job_1789307359180_0006
INFO mapreduce.Job: Job job_1789307359180_0006 running in uber mode : false
INFO mapreduce.Job:  map 0% reduce 0%
INFO mapreduce.Job:  map 50% reduce 0%
INFO mapreduce.Job:  map 100% reduce 0%
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_1789307359180_0006 completed successfully
```

Job Counters (selected):

Map input records	319,237
Map output records	319,235
Launched map tasks	2 (one per input file: books.csv, reviews_100k.csv)
Reduce input groups	219,235
Reduce output records	42,345
Bytes Written	2,008,527

Results

42,345 books received at least one review in this sample — matching "Reduce output records" in the job counters exactly. The table below shows the first rows in book_id order (matching the reduce output's natural key ordering); the full result is in the project's output/ folder and in Figure 4.3.

Book ID	Title	Review Count	Avg. Review Rating
10000063	The Two Deaths of Daniel Hayes	5	3.000
10000114	The Devil's Graveyard	1	5.000
10000887	Undercover Blues (Undercover Blues, #1)	1	5.000
10001392	Only Living Witness	1	0.000
10001806	Villain	1	3.000
1000230	Cold Fire	1	2.000
1000238	The Door to December	1	4.000
10002765	Dumb Witness (Hercule Poirot, #16)	1	4.000
10003203	Cut Thin to Win	1	4.000
10003241	Lucidity	1	5.000
⋮	⋮	⋮	⋮

```

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\books-reviews-lab>hdfs dfs -cat /lab/output/task3/part-r-00000 | more
10000063 The Two Deaths of Daniel Hayes 5 3.000
10000114 The Devil's Graveyard 1 5.000
10000887 Undercover Blues (Undercover Blues, #1) 1 5.000
10001392 Only Living Witness 1 0.000
10001806 Villain 1 3.000
1000230 Cold Fire 1 2.000
1000238 The Door to December 1 4.000
10002765 Dumb Witness (Hercule Poirot, #16) 1 4.000
10003203 Cut Thin to Win 1 4.000
10003241 Lucidity 1 5.000
10003250 Three Times Three, Mystery omnibus 1 3.000
10003945 The Hypnotist 1 3.000
10006260 A Mortal Terror (Billy Boyle World War II, #6) 1 4.000
10008056 Journal 64 (Afdeling Q, #4) 1 4.000
1001018 The Players and The Game 1 5.000
10010895 The Honourable Schoolboy (The Karla Trilogy, #2) 2 4.000
10012850 Cupid's Revenge (Chloe Boston Mystery #5) 1 3.000
10013430 Outlaw Mountain (Joanna Brady, #7) 1 5.000
1001555 The Hunt for Red October (Jack Ryan Universe, #4) 2 4.000
1001556 The Firm 2 4.000
1001567 Executive Orders (Jack Ryan Universe, #9) 1 5.000
10020182 Verwesung (David Hunter, #4) 1 2.000
10020810 Tysteren (Anton Brekke #1) 1 1.000
10021227 Bet Your Bones (A Dinah Pelerin Mystery, #2) 1 3.000
10021759 Unbelievable (Pretty Little Liars, #4) 1 4.000
1002276 Life for Death 1 0.000
10023155 Timecaster 4 3.500
10023183 A Woman of Consequence (A Dido Kent Mystery #3) 2 4.000
10023746 A Pocketful of Eyes 4 3.750
10024937 Iron House 12 3.917
1002539 Interred with Their Bones (Kate Stanley, #1) 20 2.950
1002542 Midnight in the Garden of Good and Evil 4 3.750
10027945 Tylko martwi nie kłóć się (Hubert Meyer, #2) 1 1.000
100296 Murder at Willow Slough 1 2.000
10031164 Reginan laulu 1 1.000
1003167 A Test Of Wills (Inspector Ian Rutledge, #1) 1 4.000
1003292 Final Friends Trilogy 1 5.000
10033635 Blood Heat (Dangerous Ground, #3) 2 3.500
1003447 Both Ends of the Night (Sharon McCone, #18) 1 4.000
1003463 Double (Nameless Detective, #13) 1 5.000
1003464 Edwin of the Iron Shoes 3 4.333

```

Figure 4.3 — `hdfs dfs -cat` output for Book-Review Join

Note: All 42,345 books that received at least one review in this sample were found in `books.csv` — a 100% join match, with no orphaned reviews. Rating 0 ("shelved but not rated" on Goodreads) is included as-is in this average — see book 10001392, "Only Living Witness," whose single review has rating 0, giving it an average of exactly 0.000.

5. Pig Analysis

5.1 Task 1: Books by Language

Book count and average rating per language_code, top 15 languages

Script (task1_books_by_language.pig)

```
-- Pig Task 1: Number of books and average rating per language_code.
-- Input : books.csv
-- Output: pig_output/task1_books_by_language (language_code, num_books, avg_rating)

REGISTER 'file:///C:/hadoop/pig/lib/piggybank.jar'; -- adjust: run `find / -iname "piggybank*.jar"
2>/dev/null` to locate it

books = LOAD 'books.csv'
        USING org.apache.pig.piggybank.storage.CSVExcelStorage(',', 'NO_MULTILINE', 'UNIX',
'SKIP_INPUT_HEADER')
        AS (book_id:chararray, title:chararray, authors:chararray, average_rating:double,
        ratings_count:long, publication_year:chararray, language_code:chararray, isbn:chararray);

books_with_lang = FILTER books BY language_code IS NOT NULL AND language_code != '';

grouped_by_lang = GROUP books_with_lang BY language_code;

lang_stats = FOREACH grouped_by_lang GENERATE
    group AS language_code,
    COUNT(books_with_lang) AS num_books,
    ROUND(AVG(books_with_lang.average_rating) * 100.0) / 100.0 AS avg_rating;

lang_stats_ordered = ORDER lang_stats BY num_books DESC;

top_languages = LIMIT lang_stats_ordered 15;

STORE top_languages INTO 'pig_output/task1_books_by_language' USING PigStorage(',');
```

Execution

Command: pig pig-scripts/task1_books_by_language.pig

Script executed in mapreduce mode against the local dataset and completed successfully. Output was written to task1_books_by_language/part-r-00000.

Results

Language Code	Book Count	Avg. Rating
eng	80,214	3.88
en-US	11,986	3.87
en-GB	7,142	3.83
ita	5,529	3.77
ger	4,547	3.79
spa	3,900	3.80
nl	2,684	3.69

Language Code	Book Count	Avg. Rating
fre	2,562	3.80
por	2,286	3.81
swe	1,674	3.68
ind	1,495	3.84
fin	1,320	3.68
gre	1,135	3.78
en-CA	1,030	3.97
cze	1,000	3.82

```

Directory of C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset\pig_output\task1_books_by_language
09/13/2026  08:44 PM  <DIR>      .
09/13/2026  08:44 PM  <DIR>      ..
09/13/2026  08:44 PM                12  .part-r-00000.crc
09/13/2026  08:44 PM                8  .SUCCESS.crc
09/13/2026  08:44 PM               215  part-r-00000
09/13/2026  08:44 PM                0  _SUCCESS
                4 File(s)          235 bytes
                2 Dir(s)  40,354,889,728 bytes free

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset>type pig_output\task1_books_by_language\part-r-00000
eng,80214,3.88
en-US,11986,3.87
en-GB,7142,3.83
ita,5529,3.77
ger,4547,3.79
spa,3900,3.8
nl,2684,3.69
fre,2562,3.8
por,2286,3.81
swe,1674,3.68
ind,1495,3.84
fin,1320,3.68
gre,1135,3.78
en-CA,1030,3.97
cze,1000,3.82

```

Figure 5.1 — output for Books by Language

Note: language_code is missing for about 38% of books.csv; those rows are excluded here rather than counted as a spurious "unknown" language — see Section 6.

5.2 Task 2: Popular & Highly Rated Books

Books with average_rating ≥ 4.5 and ratings_count ≥ 1,000, top 20 by rating count

Script (task2_popular_highly_rated_books.pig)

```

-- Pig Task 2: Highly-rated, popular books.
-- Filters for average_rating >= 4.5 AND ratings_count >= 1000, then keeps
-- the 20 books with the most ratings.
-- Input : books.csv
-- Output: pig_output/task2_popular_highly_rated_books (book_id, title, average_rating, ratings_count)

REGISTER 'file:///C:/hadoop/pig/lib/piggybank.jar';  -- adjust: run `find / -iname "piggybank*.jar"
2>/dev/null` to locate it

books = LOAD 'books.csv'
        USING org.apache.pig.piggybank.storage.CSVExcelStorage(' ', 'NO_MULTILINE', 'UNIX',
        'SKIP_INPUT_HEADER')
        AS (book_id:chararray, title:chararray, authors:chararray, average_rating:double,
            ratings_count:long, publication_year:chararray, language_code:chararray, isbn:chararray);

```

```

popular_books = FILTER books BY (average_rating >= 4.5) AND (ratings_count >= 1000);

popular_sorted = ORDER popular_books BY ratings_count DESC;

top20 = LIMIT popular_sorted 20;

result = FOREACH top20 GENERATE book_id, title, average_rating, ratings_count;

STORE result INTO 'pig_output/task2_popular_highly_rated_books' USING PigStorage(',');

```

Execution

Command: pig pig-scripts/task2_popular_highly_rated_books.pig

Script executed in mapreduce mode against the local dataset and completed successfully. Output was written to task2_popular_highly_rated_books/part-r-00000.

Results

Book ID	Title	Avg. Rating	Ratings Count
188572	The Complete Sherlock Holmes	4.50	111,473
10147	The Complete Sherlock Holmes, Volume I	4.50	16,821
228199	The Green Mile, Part 4: The Bad Death of Eduard Delacroix	4.52	13,164
228198	The Green Mile, Part 3: Coffey's Hands	4.51	12,556
71823	The Green Mile, Part 6: Coffey on the Mile	4.55	12,215
228200	The Green Mile, Part 5: Night Journey	4.52	11,679
892295	The Annotated Sherlock Holmes: The Four Novels and the Fifty-Six Short Stories Complete (2 Volume Set)	4.58	9,672
12384681	Divide & Conquer (Cut & Run, #4)	4.50	8,734
13246997	Armed & Dangerous (Cut & Run, #5)	4.52	8,563
14061520	Stars & Stripes (Cut & Run, #6)	4.59	7,865
16159276	Touch & Geaux (Cut & Run, #7)	4.52	7,152
33533363	High Force (DCI Ryan Mysteries, #5)	4.52	4,499
9507278	The Complete Works of Edgar Allan Poe	4.52	3,845
16566323	The Complete Sherlock Holmes	4.50	3,199
18881367	Fatal Jeopardy (Fatal, #7)	4.50	3,086
26789784	Hurricane Force (Miss Fortune Mystery #7)	4.51	2,988
29915419	Fortune Hunter (Miss Fortune Mystery #8)	4.52	2,722
21468248	Fatal Scandal (Fatal, #8)	4.54	2,722
33142914	Later Gator (Miss Fortune Mystery #9)	4.54	2,330
32195800	Pretty Lost Dolls (Pretty Little Dolls, #2)	4.52	2,304

```

Directory of C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset\pig_output\task2_popular_highly_rated_books

09/13/2026  08:49 PM  <DIR>          .
09/13/2026  08:49 PM  <DIR>          ..
09/13/2026  08:49 PM                20 .part-r-00000.crc
09/13/2026  08:49 PM                8 .SUCCESS.crc
09/13/2026  08:49 PM            1,164 part-r-00000
09/13/2026  08:49 PM                0 SUCCESS
                4 File(s)            1,192 bytes
                2 Dir(s)  40,352,047,104 bytes free

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset>type pig_output\task2_popular_highly_rated_books\part-r-00000
188572,The Complete Sherlock Holmes,4.5,111473
10147,The Complete Sherlock Holmes, Volume I,4.5,16821
228199,The Green Mile, Part 4: The Bad Death of Eduard Delacroix,4.52,13164
228198,The Green Mile, Part 3: Coffey's Hands,4.51,12556
71823,The Green Mile, Part 6: Coffey on the Mile,4.55,12215
228200,The Green Mile, Part 5: Night Journey,4.52,11679
892295,The Annotated Sherlock Holmes: The Four Novels and the Fifty-Six Short Stories Complete (2 Volume Set),4.58,9672
12384681,Divide & Conquer (Cut & Run, #4),4.5,8734
13246997,Armed & Dangerous (Cut & Run, #5),4.52,8563
14061520,Stars & Stripes (Cut & Run, #6),4.59,7865
16159276,Touch & Geaux (Cut & Run, #7),4.52,7152
33533363,High Force (DCI Ryan Mysteries, #5),4.52,4499
9507278,The Complete Works of Edgar Allan Poe,4.52,3845
16566323,The Complete Sherlock Holmes,4.5,3199
18881367,Fatal Jeopardy (Fatal, #7),4.5,3086
26789784,Hurricane Force (Miss Fortune Mystery #7),4.51,2988
21468248,Fatal Scandal (Fatal, #8),4.54,2722
29915419,Fortune Hunter (Miss Fortune Mystery #8),4.52,2722
33142914,Later Gator (Miss Fortune Mystery #9),4.54,2330
32195800,Pretty Lost Dolls (Pretty Little Dolls, #2),4.52,2304

```

Figure 5.2 — output for Popular & Highly Rated Books

Note: The two Green Mile serial installments and multiple Complete Sherlock Holmes editions each count as separate books here, since they have distinct book_id values despite overlapping content.

5.3 Task 3: Rating Histogram

Same question as MapReduce Task 1 — included as a cross-check

Script (task3_rating_histogram.pig)

```

-- Pig Task 3: Review rating histogram (how many reviews gave each star rating).
-- Intentionally mirrors Java Task 1 (RatingHistogram) so you can compare
-- the Pig and MapReduce results and confirm they match.
-- Input : reviews_100k.csv
-- Output: pig_output/task3_rating_histogram (rating, num_reviews)

REGISTER 'file:///C:/hadoop/pig/lib/piggybank.jar'; -- adjust: run `find / -iname "piggybank*.jar"
2>/dev/null` to locate it

reviews = LOAD 'reviews_100k.csv'
    USING org.apache.pig.piggybank.storage.CSVExcelStorage(',', 'NO_MULTILINE', 'UNIX',
'SKIP_INPUT_HEADER')
    AS (user_id:chararray, book_id:chararray, review_id:chararray, rating:int,
review_text:chararray,
    date_added:chararray, date_updated:chararray, read_at:chararray, started_at:chararray,
n_votes:int, n_comments:int);

grouped_by_rating = GROUP reviews BY rating;

histogram = FOREACH grouped_by_rating GENERATE group AS rating, COUNT(reviews) AS num_reviews;

histogram_ordered = ORDER histogram BY rating ASC;

STORE histogram_ordered INTO 'pig_output/task3_rating_histogram' USING PigStorage(',');

```

Execution

Command: `pig pig-scripts/task3_rating_histogram.pig`

Script executed in mapreduce mode against the local dataset and completed successfully. Output was written to `task3_rating_histogram/part-r-00000`.

Results

Rating	Number of Reviews
0	2,488
1	2,903
2	7,927
3	23,082
4	35,659
5	27,941

```
Directory of C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset\pig_output\task3_rating_histogram
09/13/2026  08:53 PM  <DIR>      .
09/13/2026  08:53 PM  <DIR>      ..
09/13/2026  08:53 PM                12 .part-r-00000.crc
09/13/2026  08:53 PM                8 ._SUCCESS.crc
09/13/2026  08:53 PM               45 part-r-00000
09/13/2026  08:53 PM                0 _SUCCESS
                4 File(s)              65 bytes
                2 Dir(s)  40,348,352,512 bytes free

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset>type pig_output\task3_rating_histogram\part-r-00000
0,2488
1,2903
2,7927
3,23082
4,35659
5,27941
```

Figure 5.3 — output for Rating Histogram

Note: This result matches MapReduce Task 1 (§4.1) exactly — same six counts — confirming both pipelines read and aggregate reviews_100k.csv consistently.

5.4 Task 4: Book–Review Join

Same question as MapReduce Task 3, via a single Pig JOIN, filtered to books with at least 20 reviews, top 25 by average review rating

Script (`task4_book_review_join.pig`)

```
-- Pig Task 4: Join books.csv with reviews_100k.csv on book_id to find books
-- that are both heavily discussed and well-liked by reviewers.
-- Intentionally mirrors Java Task 3 (BookReviewJoin) so you can compare a
-- one-line Pig JOIN against the equivalent hand-written reduce-side join.
-- Input : books.csv, reviews_100k.csv
-- Output: pig_output/task4_book_review_join (book_id, title, review_count, avg_review_rating)

REGISTER 'file:///C:/hadoop/pig/lib/piggybank.jar'; -- adjust: run `find / -iname "piggybank*.jar"
2>/dev/null` to locate it

books = LOAD 'books.csv'
        USING org.apache.pig.piggybank.storage.CSVExcelStorage(',', 'NO_MULTILINE', 'UNIX',
'SKIP_INPUT_HEADER')
        AS (book_id:chararray, title:chararray, authors:chararray, average_rating:double,
```

```

        ratings_count:long, publication_year:chararray, language_code:chararray, isbn:chararray);

reviews = LOAD 'reviews_100k.csv'
        USING org.apache.pig.piggybank.storage.CSVExcelStorage(',', 'NO_MULTILINE', 'UNIX',
'SKIP_INPUT_HEADER')
        AS (user_id:chararray, book_id:chararray, review_id:chararray, rating:int,
review_text:chararray,
        date_added:chararray, date_updated:chararray, read_at:chararray, started_at:chararray,
n_votes:int, n_comments:int);

joined = JOIN books BY book_id, reviews BY book_id;

grouped_by_book = GROUP joined BY books::book_id;

book_review_stats = FOREACH grouped_by_book GENERATE
        group AS book_id,
        MAX(joined.books::title) AS title,
        COUNT(joined) AS review_count,
        ROUND(AVG(joined.reviews::rating) * 100.0) / 100.0 AS avg_review_rating;

popular_reviewed = FILTER book_review_stats BY review_count >= 20;

result_sorted = ORDER popular_reviewed BY avg_review_rating DESC, review_count DESC;

top25 = LIMIT result_sorted 25;

STORE top25 INTO 'pig_output/task4_book_review_join' USING PigStorage(',');

```

Execution

Command: pig pig-scripts/task4_book_review_join.pig

Script executed in mapreduce mode against the local dataset and completed successfully. Output was written to task4_book_review_join/part-r-00000.

Results

Book ID	Title	Review Count	Avg. Rating
16070018	Truth (Consequences, #2)	67	4.72
16159276	Touch & Geaux (Cut & Run, #7)	20	4.60
18052619	Killing Sarai (In the Company of Killers, #1)	35	4.54
25812109	The Female of the Species	28	4.54
8701129	Love You More (Tessa Leoni, #1; Detective D.D. Warren, #5)	20	4.45
13642963	Speaking from Among the Bones (Flavia de Luce, #5)	25	4.44
16150996	Convicted (Consequences, #3)	59	4.39
32423	Watchers	23	4.39
9970042	Whispers Under Ground (Peter Grant, #3)	23	4.39
22929578	The Fixer (The Fixer, #1)	23	4.39
23807	The Silence of the Lambs (Hannibal Lecter, #2)	24	4.38
29981261	The Butterfly Garden (The Collector #1)	32	4.34

Book ID	Title	Review Count	Avg. Rating
28220985	A Great Reckoning (Chief Inspector Armand Gamache, #12)	22	4.32
9226039	Into the Darkest Corner	32	4.31
32075854	Magpie Murders	23	4.30
26082188	The Dispatcher	20	4.30
19486412	Big Little Lies	214	4.29
20980959	Blue Labyrinth (Pendergast, #14)	21	4.29
18090147	The Enchanted	32	4.28
10680329	A Trick of the Light (Chief Inspector Armand Gamache, #7)	25	4.28
29430013	The Trespasser (Dublin Murder Squad #6)	30	4.27
25735012	Career of Evil (Cormoran Strike, #3)	88	4.26
12922320	Criminal (Will Trent, #6)	23	4.26
33561990	Bad Mommy	23	4.26
15729539	NOS4A2	95	4.25

```

Directory of C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset\pig_output\task4_book_review_join
09/13/2026  08:59 PM  <DIR>          .
09/13/2026  08:59 PM  <DIR>          ..
09/13/2026  08:59 PM                20 .part-r-00000.crc
09/13/2026  08:59 PM                8  _SUCCESS.crc
09/13/2026  08:59 PM            1,194 part-r-00000
09/13/2026  08:59 PM                0  _SUCCESS
                4 File(s)          1,222 bytes
                2 Dir(s)  40,345,718,784 bytes free

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset>type pig_output\task4_book_review_join\part-r-00000
16070018,Truth (Consequences, #2),67,4.72
16159276,Touch & Geaux (Cut & Run, #7),20,4.6
18052619,Killing Sarai (In the Company of Killers, #1),35,4.54
25812109,The Female of the Species,28,4.54
8701129,Love You More (Tessa Leoni, #1; Detective D.D. Warren, #5),20,4.45
13642963,Speaking from Among the Bones (Flavia de Luce, #5),25,4.44
16150996,Convicted (Consequences, #3),59,4.39
32423,Watchers,23,4.39
9970042,Whispers Under Ground (Peter Grant, #3),23,4.39
22929578,The Fixer (The Fixer, #1),23,4.39
23807,The Silence of the Lambs (Hannibal Lecter, #2),24,4.38
29981261,The Butterfly Garden (The Collector #1),32,4.34
28220985,A Great Reckoning (Chief Inspector Armand Gamache, #12),22,4.32
9226039,Into the Darkest Corner,32,4.31
32075854,Magpie Murders,23,4.3
26082188,The Dispatcher,20,4.3
19486412,Big Little Lies,214,4.29
20980959,Blue Labyrinth (Pendergast, #14),21,4.29
18090147,The Enchanted,32,4.28
10680329,A Trick of the Light (Chief Inspector Armand Gamache, #7),25,4.28
29430013,The Trespasser (Dublin Murder Squad #6),30,4.27
25735012,Career of Evil (Cormoran Strike, #3),88,4.26
12922320,Criminal (Will Trent, #6),23,4.26
33561990,Bad Mommy,23,4.26
15729539,NOS4A2,95,4.25

```

Figure 5.4 — output for Book–Review Join

Note: Filtering to `review_count ≥ 20` keeps single-review outliers (like the 0.000 and 5.000 averages seen in the unfiltered MapReduce Task 3 result) out of this leaderboard, so every book shown here has a statistically meaningful number of reviews behind its average.

5.5 Task 5: Average Rating by Publication Decade

Average books.csv rating and book count, grouped by decade

Script (task5_avg_rating_by_decade.pig)

```
-- Pig Task 5: Average book rating and book count per publication decade.
-- Input : books.csv
-- Output: pig_output/task5_avg_rating_by_decade (decade, num_books, avg_rating)

REGISTER 'file:///C:/hadoop/pig/lib/piggybank.jar'; -- adjust: run `find / -iname "piggybank*.jar"
2>/dev/null` to locate it

books = LOAD 'books.csv'
        USING org.apache.pig.piggybank.storage.CSVExcelStorage(',', 'NO_MULTILINE', 'UNIX',
'SKIP_INPUT_HEADER')
        AS (book_id:chararray, title:chararray, authors:chararray, average_rating:double,
            ratings_count:long, publication_year:chararray, language_code:chararray, isbn:chararray);

valid_year_books = FILTER books BY publication_year IS NOT NULL
                        AND publication_year != ''
                        AND publication_year MATCHES '\\d{4}';

years_as_int = FOREACH valid_year_books GENERATE
                book_id, average_rating, (int)publication_year AS pub_year;

with_decade = FOREACH years_as_int GENERATE
                book_id, average_rating, ((int)(pub_year / 10)) * 10 AS decade;

grouped_by_decade = GROUP with_decade BY decade;

decade_stats = FOREACH grouped_by_decade GENERATE
                group AS decade,
                COUNT(with_decade) AS num_books,
                ROUND(AVG(with_decade.average_rating) * 100.0) / 100.0 AS avg_rating;

decade_stats_ordered = ORDER decade_stats BY decade ASC;

STORE decade_stats_ordered INTO 'pig_output/task5_avg_rating_by_decade' USING PigStorage(',');
```

Execution

Command: pig pig-scripts/task5_avg_rating_by_decade.pig

Script executed in mapreduce mode against the local dataset and completed successfully. Output was written to task5_avg_rating_by_decade/part-r-00000.

Results

Decade	Book Count	Avg. Rating
1210	1	4.02
1390	6	3.92
1830	2	3.55
1840	1	4.22
1850	1	3.71
1880	1	4.15

Decade	Book Count	Avg. Rating
1890	14	3.74
1900	26	3.77
1910	30	3.58
1920	101	3.60
1930	205	3.75
1940	385	3.72
1950	785	3.69
1960	1,431	3.75
1970	2,361	3.78
1980	5,429	3.78
1990	13,512	3.78
2000	42,721	3.78
2010	97,627	3.84
2020	3	4.30
2030	1	3.66
2060	1	3.75
2100	14	4.00
2110	1	3.58
2140	1	4.11
2200	1	3.89
2910	1	4.00
3010	2	4.02

```

Directory of C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset\pig_output\task5_avg_rating_by_decade

09/13/2026  09:02 PM    <DIR>        .
09/13/2026  09:02 PM    <DIR>        ..
09/13/2026  09:02 PM                12 .part-r-00000.crc
09/13/2026  09:02 PM                8 ._SUCCESS.crc
09/13/2026  09:02 PM            365 part-r-00000
09/13/2026  09:02 PM                0 ._SUCCESS
         4 File(s)            385 bytes
         2 Dir(s)  40,341,348,352 bytes free

C:\Users\ABRAR\OneDrive\Desktop\BigDataLabFinal\dataset>type pig_output\task5_avg_rating_by_decade\part-r-00000
1210,1,4.02
1390,6,3.92
1830,2,3.55
1840,1,4.22
1850,1,3.71
1880,1,4.15
1890,14,3.74
1900,26,3.77
1910,30,3.58
1920,101,3.6
1930,205,3.75
1940,385,3.72
1950,785,3.69
1960,1431,3.75
1970,2361,3.78
1980,5429,3.78
1990,13512,3.78
2000,42721,3.78
2010,97627,3.84
2020,3,4.3
2030,1,3.66
2060,1,3.75
2100,14,4.0
2110,1,3.58
2140,1,4.11
2200,1,3.89
2910,1,4.0
3010,2,4.02

```

Figure 5.5 — output for Average Rating by Publication Decade

Note: Pig's ROUND() rounds half-up (it follows Java's Math.round()), which differs from spreadsheet/Python round-half-to-even behavior on exact .5 ties. The 1830s decade (2 books, raw average exactly 3.545) rounds to 3.55 under this convention — not an error, just a different tie-breaking rule than some tools default to. See Section 6 for the decade groupings built from implausible publication_year values (1210, 2910, 3010).

6. Data Quality Notes

A few data-quality issues in the raw books.csv metadata are worth noting, since they affect several of the results above:

- **Missing publication_year:** publication_year is missing for ~25% of books.csv (54,494 of 219,235 rows) — those rows are excluded from Task 2 (MapReduce, §4.2) and Pig Task 5 (§5.5) rather than defaulting to 0, which would have skewed every year's or decade's average downward.
- **Missing language_code:** language_code is missing for ~38% of books.csv (82,451 rows) — excluded from Pig Task 1 (§5.1) for the same reason.
- **Implausible publication years:** A handful of books.csv rows have a publication_year that is technically a valid 4-digit number but not a plausible year — most notably 3013 (2 books: "Twisting Steele" and "Dylan's Monster"), 2911 (1 book: "Trial by Ambush"), and 1213 (1 book: "So Say the Waiters book 2"). Because the mapper's validity check only confirms the value is exactly 4 digits, these pass through into Pig Task 5's decade groupings, producing the isolated 3010, 2910, and 1210 rows seen in that result (§5.5). These are very likely scraping or data-entry errors in the source metadata, affecting only 4 of 164,741 valid-year rows (about 0.002%).
- **A likely calendar-system mismatch:** A related but likely legitimate case: six books.csv rows with Persian-language titles (e.g. "دختری در قطار" / The Girl on the Train, "جایی برای پیرمردها نیست" / No Country for Old Men) carry publication years of 1393–1396. These line up closely with the Persian (Solar Hijri) calendar equivalent of 2014–2017 — right around when those books' English originals were released — suggesting this cluster reflects the Persian edition's publication year recorded in a different calendar system, rather than data corruption.
- **rating = 0 is not a real zero-star rating:** rating = 0 in reviews_100k.csv denotes "shelved / read but not rated" on Goodreads, not a genuine zero-star review. MapReduce Task 3 (§4.3) and Pig Task 4 (§5.4) include these as-is in their averages — see the "Only Living Witness" example in §4.3, where a single rating-0 review gives that book an average of exactly 0.000.
- **Pig rounding convention:** Pig's ROUND() rounds half-up rather than round-half-to-even (the default in Python and most spreadsheets) — see §5.5 for a worked example using the 1830s decade, where this changes the displayed average from 3.54 to 3.55.

7. Conclusion

All three MapReduce jobs and all five Pig scripts were implemented and executed against the 219,235-book, 100,000-review Goodreads dataset sample. Cross-checks between independently written jobs — the MapReduce and Pig rating histograms matching exactly (§4.1, §5.3), and the Book–Review Join achieving 100% book-to-metadata coverage in both the MapReduce and Pig implementations (§4.3, §5.4) — confirm that both pipelines read and aggregate the dataset consistently. Several data-quality issues in the source metadata — missing publication years and language codes, a small number of implausible publication years, and a likely non-Gregorian-calendar cluster — were identified and documented in Section 6, and were handled by exclusion rather than by silently defaulting to zero, to avoid skewing any of the averages reported above.